UCAM · INGENIERÍA INFORMÁTICA

Desarrollo de Aplicaciones Distribuidas I

Resumen del temario para el primer parcial

Hilos · Sockets · IPC · HTTP · FTP · Plantillas de examen

Cómo usar esta guía

- **Lee en orden** (1 → 5): cada tema se apoya en el anterior. Al final de cada tema tienes un recuadro **«Para el examen»**.
- El examen real es sobre todo **programación**: dedica la mitad del tiempo al capítulo 6 (receta + código) y a escribir el servidor **a mano, sin mirar**.
- El capítulo 7 recoge los fallos de hilos y sockets que han caído; el 8 es una autoevaluación con respuestas.
- Todo el código de la guía está compilado y probado.

Índice

1 Computación distribuida
2 Comunicación entre procesos (IPC)
3 Hilos de ejecución en Java
4 Sockets
5 Protocolos: HTTP y FTP
6 El servidor multihilo: receta y plantillas
7 Errores típicos que caen en el examen
8 Autoevaluación (con respuestas)
9 Checklist del día antes

Mapa del temario



Tema	Idea central	Lo que te piden
1 Distribuida	Computadores independientes en red que colaboran	Definición, ventajas, formas de computación
2 IPC	Procesos que se comunican: enviar/recibir, síncrono/asíncrono	Bloqueo indefinido y sus 2 soluciones
3 Hilos	Varias tareas a la vez dentro de un programa	**Escribir y lanzar hilos**, errores con start() y sincronización
4 Sockets	Punto de conexión = IP + puerto	**Servidor y cliente en Java**
5 HTTP/FTP	Protocolos de texto petición-respuesta	Características y estructura de mensaje, métodos, códigos

1 Computación distribuida

Tema 1.1

Qué es un sistema distribuido

> **Definición**
> Conjunto de computadores **independientes**, interconectados a través de una **red**, capaces de **colaborar** para realizar una tarea. **Independientes** = no comparten memoria ni espacio de ejecución.

- **Monoprocesador**: una única CPU (el modelo anterior).
- **Computación distribuida**: la computación que se lleva a cabo en un sistema distribuido.
- **Servicio de red**: lo ofrece un programa especial (el **servidor**): WWW, FTP, POP3.
- **Aplicación de red**: aplicación de usuario final que corre en ordenadores conectados.

Historia (por si cae)

- Años 60: primera conexión para compartir ficheros · 1972: primer e-mail en ARPANET · 1985: FTP · finales de los 80: WWW.
- Los protocolos se documentan en **RFC** (Request For Comments).

Ventajas e inconvenientes

Ventajas	Inconvenientes
Económica (computadores y red baratos) **Compartición de recursos** **Escalabilidad** **Tolerancia a fallos**	**Múltiples puntos de fallo** **Seguridad**

Formas de computación

Forma	Descripción
Monolítica	Un computador sin conexión y un usuario. (*Mainframe*: varios usuarios, cada uno con su sesión).
Distribuida	Muchos computadores conectados por red.
Paralela	Varios procesadores a la vez para un programa. Difícil dividir el programa. (Biología, aeronáutica, IA).
Cooperativa	Repartir la computación entre ordenadores para aprovechar ciclos de CPU sobrantes.

Para el examen

«Ventajas de la computación distribuida» (1 punto en el parcial): **económica, compartir recursos, escalable, tolerante a fallos**. Memoriza también los 2 inconvenientes. **Distribuida = muchos computadores; paralela = varios procesadores.**

2 Comunicación entre procesos (IPC)

Temas 1.2 y 2.1

Idea

Un sistema distribuido **es** comunicación entre procesos. Dos procesos independientes intercambian datos (emisor → receptor) y para ello necesitan un **protocolo**: ¿quién es emisor, quién receptor, qué formato?

- **Unicast**: de un proceso a otro. **Multicast**: de un proceso a varios.
- El SO ofrece colas de mensajes, semáforos y memoria compartida; se suele usar una **capa de abstracción** encima (en Java, los sockets).

Operaciones primitivas

Operación	Significado
Enviar	Emisor: transmite datos.
Recibir	Receptor: los recoge.
Conectar	Uno invoca *solicitar-conexión*, el otro *aceptar-conexión*.
Desconectar	Libera ambos extremos.

Cada proceso las invoca de forma **independiente**; cada invocación produce un **evento**.

Sincronización: bloqueante o no

Regla

Bloqueante = síncrono (espera a que termine). **No bloqueante = asíncrono** (sigue sin esperar). Ejemplo: la petición web es síncrona, la respuesta del servidor asíncrona.

Enviar	Recibir	Qué ocurre
síncrono	síncrono	Ambos esperan; la librería IPC da un asentimiento transparente.
asíncrono	síncrono	El emisor sigue; el receptor se bloquea hasta que llegan los datos.
síncrono	asíncrono	Escenario 1: asentimiento normal. **Escenario 2: el recibir ya retornó vacío y el envío bloqueante nunca se desbloquea → bloqueo indefinido.** Escenario 3: se notifica la llegada de datos.
asíncrono	asíncrono	Nadie se bloquea; se notifica al receptor cuando llegan los datos.

Bloqueo indefinido y cómo evitarlo

Ocurre cuando una petición espera una operación que **nunca termina**. Dos soluciones:

- **Temporizadores (timers)**: límite de tiempo de espera; si no llega la respuesta, se abandona.
- **Proceso hijo o hilo de ejecución**: otra tarea hace la espera mientras el programa sigue (conexiones de red, escritura a disco).

Para el examen

Pregunta típica: «bloqueo de acceso y sus 2 soluciones» → **timers + hilos**. Recuerda también el **estado**: si el servidor guarda información de comunicaciones anteriores, la conexión tiene estado (HTTP no lo tiene → cookies).

3 Hilos de ejecución en Java

Taller + proyecto «Hilos de ejecución»

Definición

Hilo: secuencia única de control de flujo dentro de un programa; la unidad de código más pequeña que se puede ejecutar.

Para qué sirven: tareas lentas · no paralizar el programa principal · evitar bloqueos inaceptables · hacer varias tareas a la vez (p. ej. un servidor atendiendo a varios clientes).

Dos formas de crear un hilo

	Heredando de Thread	Implementando Runnable
Clase	`class MiHilo extends Thread`	`class MiHilo implements Runnable`
Método	`public void run()`	`public void run()`
Lanzar	`new MiHilo().start()`	`new Thread(new MiHilo()).start()`

```
public class MiHilo extends Thread {
    @Override
    public void run() {
        for (int i = 0; i < 5; i++) {
            System.out.println("Ejecución: " + i);
            try { Thread.sleep(500); } catch (InterruptedException e) { e.printStackTrace(); }
        }
    }
}
// main:  MiHilo h = new MiHilo();  h.start();   // main y el hilo avanzan EN PARALELO
```

Métodos clave

Método	Para qué
`start()`	Crea el hilo y ejecuta **run()** en paralelo. Solo se puede llamar **una vez** por objeto.
`run()`	El código del hilo. Llamarlo directamente **no crea hilo**: corre en el hilo actual.
`Thread.sleep(ms)`	Duerme el hilo actual. Obliga a capturar **InterruptedException**.
`join()` / `join(ms)`	Espera a que termine otro hilo (o como máximo ms milisegundos).
`interrupt()`	Interrumpe: si el hilo duerme, salta InterruptedException.

```
UnHilo unHilo = new UnHilo();          // dentro hace sleep(10000)
unHilo.start();
unHilo.join(5000);                     // main espera como máximo 5 s
// unHilo.interrupt();                 // para forzar la parada si sigue dormido
```

Condición de carrera y synchronized

Si varios hilos modifican la **misma variable**, **contador++** (leer → sumar → escribir) no es atómico: se pisan y el resultado sale **menor y distinto en cada ejecución**. Se evita dejando **un solo hilo a la vez** en la sección crítica:

```
public class Contador {
    private int total = 0;
    public synchronized void sumaUno() {      // solo un hilo a la vez
        this.total = this.total + 1;
    }
    public int getTotal() { return this.total; }
}
// 200 hilos x 100 incrementos -> total correcto = 20000
```

Alternativa: **AtomicInteger** (`incrementAndGet()`) o un bloque `synchronized(objeto) { ... }`.

Parar un hilo desde una interfaz (taller Swing)

- El hilo tiene un booleano **running** que revisa en su bucle; un método lo pone a **false** para pararlo.
- Un **JComboBox** guarda los hilos lanzados; el botón «Lanzar» crea y arranca un **HiloContador** y lo añade al combo; «Parar» toma `getSelectedItem()` y lo detiene.
- Para quitarlo de la lista: `comboHilos.removeItem(comboHilos.getSelectedItem())`. Un contador **static** (*porDondeVoy*) da un id único a cada hilo.

> **Para el examen**
> «Lanzar hilos» cae seguro. Reglas de oro: **start() no run()** · un objeto nuevo por cada start() · si usas Runnable hay que envolverlo en **new Thread(...)** · **no se pueden crear objetos de una interfaz**.

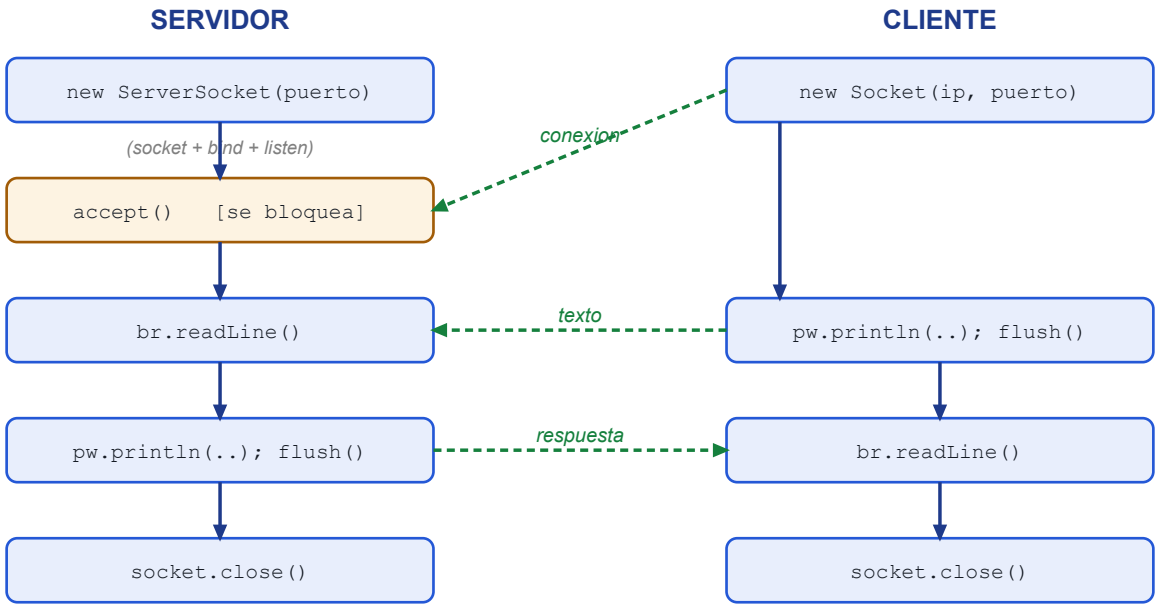
4 Sockets

Tema 2.2

Definición

Socket: punto de conexión entre dos procesos, identificado por una **IP y un puerto**. Para comunicarte necesitas un objeto **Socket** en cada extremo; los dos puertos (servidor y cliente) identifican la comunicación. Debe existir un **protocolo** acordado.

Cómo se establece la conexión



Esquema de una conversación servidor-cliente (flechas discontinuas = datos por la red).

- El servidor llama a **bind()** con un puerto elegido por él; el cliente recibe un puerto **aleatorio** del SO.
- **accept()** detiene al servidor hasta que llega un cliente: es lo que «activa» el socket.
- El servidor no necesita IP (la máquina es él); el **puerto debe coincidir** en ambos o no hay conexión.

Las dos clases de Java

Clase	Métodos clave
`Socket`	`Socket(String host, int puerto)`, `getInputStream()`, `getOutputStream()`, `close()`
`ServerSocket`	`ServerSocket(int puerto)`, `accept()` (devuelve un Socket por cada cliente)

Para leer y escribir texto se envuelven los flujos:

- `BufferedReader br = new BufferedReader(new InputStreamReader(socket.getInputStream()))` → **leer** con `br.readLine()`
- `PrintWriter pw = new PrintWriter(new OutputStreamWriter(socket.getOutputStream()))` → **escribir** con `pw.println(..); pw.flush();`
- **InputStreamReader** pasa de bytes a caracteres; **BufferedReader** permite leer líneas enteras. Sin **flush()** el otro lado no recibe nada.

Primer ejemplo completo (el que hace el profesor)

```
// ---- SERVIDOR (se arranca primero)
ServerSocket serverSocket = new ServerSocket(5050);
Socket socket = serverSocket.accept();                  // espera al cliente
PrintWriter pw = new PrintWriter(new OutputStreamWriter(socket.getOutputStream()));
BufferedReader br = new BufferedReader(new InputStreamReader(socket.getInputStream()));
String linea = br.readLine();
pw.println("Te la devuelvo " + linea); pw.flush();

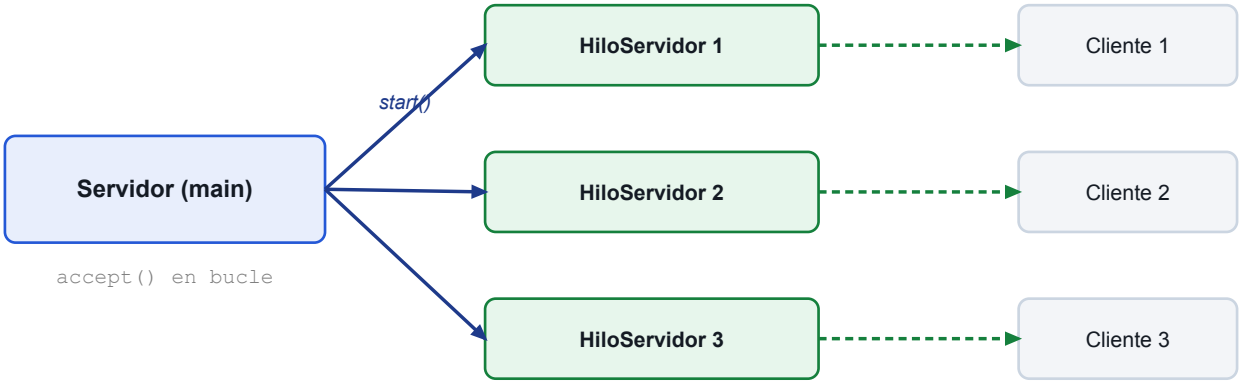
// ---- CLIENTE
Socket socket = new Socket("127.0.0.1", 5050);
PrintWriter pw = new PrintWriter(new OutputStreamWriter(socket.getOutputStream()));
BufferedReader br = new BufferedReader(new InputStreamReader(socket.getInputStream()));
pw.println("Hola!!"); pw.flush();
System.out.println(br.readLine());
```

Cuidado con el puerto

BindException: Address already in use = el puerto ya está ocupado: casi siempre **una ejecución anterior del servidor sigue viva** (nunca terminó). Párala o cambia de puerto. En macOS el puerto **5000** lo usa AirPlay: usa 5050, 8000…

Varios clientes a la vez: servidor multihilo

Con un solo hilo, el servidor se queda atendiendo al primer cliente (dentro de su bucle de lectura) y **no vuelve a accept()**. Solución: el hilo principal **solo hace accept()** y por cada cliente lanza un hilo que lleva esa conversación.



Para el examen

«¿Qué es un socket?» → punto de conexión entre dos procesos identificado por IP y puerto. «¿Cómo se activa?» → **accept()**. Si ves un servidor con un bucle de lectura y varios clientes, la respuesta es **un hilo por cliente**.

5 Protocolos: HTTP y FTP

Tema 2.3

- **Basados en texto**: se envían cadenas de caracteres (tipo ASCII).
- **Solicitud-respuesta**: un lado pide y espera la respuesta.

HTTP

> **Qué es**
> **HyperText Transfer Protocol** (RFC 1945): protocolo que permite la transferencia de información en la web entre navegadores y servidores. **No es lo mismo que HTML.**

Característica	Detalle
Basado en **URL**	Objeto al que se le pide ejecutar un método y obtener un resultado según los parámetros.
Versiones	0.9 (texto sin formato) → 1.0 (formatos MIME) → 1.1 (nuevos servicios, reinicio de comunicaciones).
Uso	Lo usa el **100 %** de usuarios y servidores.
Genérico	Permite definir tus propios comportamientos.
Sin estado	No recuerda peticiones anteriores. Primera solución: **cookies** (el servidor da un identificador aleatorio y el navegador lo devuelve).

Estructura de un mensaje HTTP

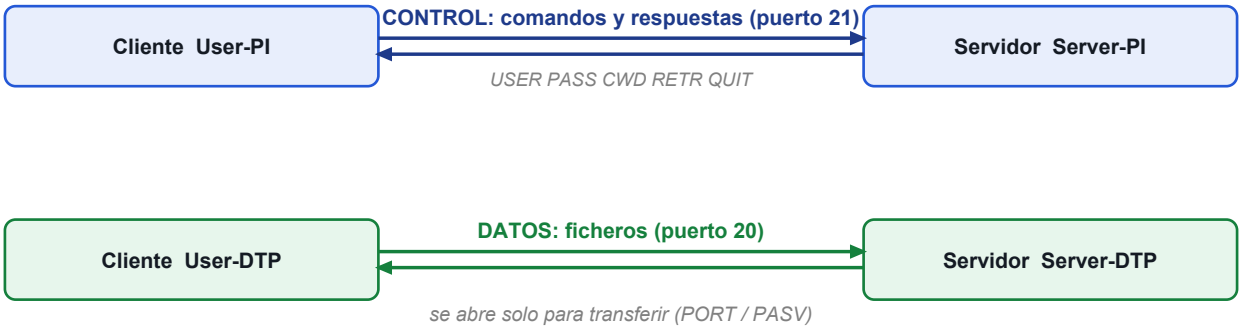
```
PETICIÓN                                    RESPUESTA
GET /pub/WWW/TheProject.html HTTP/1.0       HTTP/1.0 404 Not Found      <- primera línea
Host: www.w3.org                            Content-Type: text/html     <- cabeceras
                                            Content-Length: 120
                                                                       <- línea en blanco (CRLF)
(cuerpo opcional: datos / parámetros)       <html>...</html>           <- cuerpo
```

- **Primera línea**: petición = **método + recurso (URL) + versión**; respuesta = **versión + código de estado (3 dígitos) + frase**.
- **Cabeceras**: pares `campo:valor` acabados en CRLF (metainformación; una indica el tamaño del cuerpo).
- **Línea en blanco** (CRLF) y después el **cuerpo** (entity body).

Método	Función
GET	Recupera la URL especificada.
POST	Envía datos a la URL.
PUT	Guarda los datos en la URL.
DELETE	Borra el recurso.
HEAD / PATCH / COPY / MOVE / LINK / TRACE / OPTIONS…	Menos importantes (el profe: «no hay que aprender todos»).

FTP

> **Qué es**
> **File Transfer Protocol** (RFC 959): compartir ficheros (binarios o de datos), usar ordenadores remotos y transferir ficheros de forma eficiente. Diseñado para ser usado por **programas**.



- **PI** (Protocol Interpreter): intercambia comandos; **DTP** (Data Transfer Process): transfiere los ficheros.
- **Representación** (TYPE): **ASCII** (por defecto, texto), EBCDIC, **IMAGE** (binario, bloques de 8 bits), LOCAL.
- **Estructura** (STRU): **File** (defecto), Record, Page. **Modo** (MODE): **Stream** (defecto), Block, Compressed.

Grupo	Comandos
Acceso	`USER` nombre · `PASS` contraseña · `CWD` cambiar dir · `CDUP` subir · `QUIT` cerrar sesión y conexión · `REIN` cerrar sesión (no la conexión)
Transferencia	`PORT h1,h2,h3,h4,p1,p2` · `PASV` · `TYPE` · `STRU` · `MODE`
Servicio	`RETR` descargar · `STOR` subir · `APPE` añadir · `DELE` borrar · `MKD`/`RMD` crear/borrar dir · `PWD` · `LIST`/`NLST` · `RNFR`+`RNTO` renombrar · `NOOP`

El comando PORT

PORT h1,h2,h3,h4,p1,p2: IP = h1.h2.h3.h4 y puerto = **p1 · 256 + p2** (p1 = 8 bits de más peso). Ejemplo: puerto 4096 = 16·256 + 0 → `PORT 127,0,0,1,16,0`. Otro: `PORT 192,168,1,10,19,136` → puerto 19·256+136 = **5000**.

Código	Significado	Ejemplo
1yz	Preliminar positiva	150 Opening ASCII mode data connection
2yz	Ejecutado correctamente	200 Command okay · 230 User logged in
3yz	Intermedia positiva (falta info)	331 User name okay, need password
4yz	Negativa temporal	421 Service not available
5yz	Negativa definitiva	500 command not understood · 550 Access denied

Segundo dígito: x0z sintaxis · x1z información · x2z conexiones · x3z autenticación · x5z sistema de ficheros. **Implementación mínima**: ASCII Non-print, stream, user, quit, port, type, mode, stru, retr, stor, noop.

Para el examen

HTTP: características + estructura del paquete (1,5 puntos en el parcial) + métodos GET/POST/PUT/DELETE. FTP: **dos conexiones** (control 21, datos 20), PORT y familias de códigos.

6 El servidor multihilo: receta y plantillas

Lo que más puntos da en el examen

El ejercicio de desarrollo (5 puntos) siempre es el mismo: **un servidor al que se conectan clientes y mandan comandos de texto**. Se pide escribir el hilo que los procesa. Receta de 7 pasos:

1. **Atributos**: `Socket socket` (y, si hace falta, `Servidor servidor`); constructor que los recibe.
2. **run()** con **try/catch IOException**: crea **BufferedReader** y **PrintWriter** sobre el socket.
3. **Bucle**: `while ((linea = br.readLine()) != null)` (o hasta EXIT).
4. **Trocear**: `String[] partes = linea.split(" ")`. Si un campo lleva espacios (nombre completo): `split(" ", 3)`.
5. **switch(partes[0])** con un caso por comando + **default** («Comando no reconocido»).
6. En cada caso: **comprobar partes.length**, que el objeto **exista** y las reglas del enunciado → construir **respuesta** con el texto **exacto**.
7. `pw.println(respuesta); pw.flush();` y, al terminar, **cerrar el socket**.

Plantilla 1 - Servidor (siempre igual)

```
public class Servidor {
    public static final int PUERTO = 5050;
    public void ejecutar() {
        try {
            ServerSocket serverSocket = new ServerSocket(Servidor.PUERTO);
            System.out.println("Servidor lanzado esperando clientes...");
            while (true) {
                Socket socket = serverSocket.accept();     // se bloquea hasta que llega un cliente
                HiloServidor hilo = new HiloServidor(socket);
                hilo.start();                              // el main vuelve al accept() enseguida
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
    public static void main(String[] args) {
        (new Servidor()).ejecutar();
    }
}
```

Plantilla 2 - Hilo con switch (lo único que cambia)

```
public class HiloServidor extends Thread {
    private Socket socket;
    public HiloServidor(Socket socket) {
        this.socket = socket;
    }
    @Override
    public void run() {
        try {
            BufferedReader br = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            PrintWriter pw = new PrintWriter(new OutputStreamWriter(socket.getOutputStream()));
            boolean salir = false;
            while (!salir) {
                String linea = br.readLine();
                if (linea == null) break;                  // el cliente cerro la conexion
                String[] partes = linea.split(" ");
                String respuesta = "";
                switch (partes[0].toUpperCase()) {
                case "SUMA":
                    respuesta = "" + (Integer.parseInt(partes[1]) + Integer.parseInt(partes[2]));
                    break;
                case "RESTA":
                    respuesta = "" + (Integer.parseInt(partes[1]) - Integer.parseInt(partes[2]));
                    break;
                case "MULTIPLICA":
                    respuesta = "" + (Integer.parseInt(partes[1]) * Integer.parseInt(partes[2]));
                    break;
                case "EXIT":
                    respuesta = "Adios";
                    salir = true;
                    break;
                default:
                    respuesta = "Comando no reconocido";
                }
                pw.println(respuesta);
                pw.flush();
            }
            socket.close();
        } catch (IOException | NumberFormatException | ArrayIndexOutOfBoundsException e) {
            System.out.println("Conexion finalizada: " + e);
        }
    }
}
```

Plantilla 3 - Cliente (con teclado)

```
public class Cliente {
    public void ejecutar() {
        try (Socket socket = new Socket("127.0.0.1", Servidor.PUERTO)) {
            PrintWriter pw = new PrintWriter(new OutputStreamWriter(socket.getOutputStream()));
            BufferedReader br = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            Scanner teclado = new Scanner(System.in);
            String linea = "";
            System.out.println("Comandos: SUMA a b | RESTA a b | MULTIPLICA a b | EXIT");
            while (!linea.equalsIgnoreCase("EXIT")) {
                linea = teclado.nextLine();
                pw.println(linea);
                pw.flush();
                System.out.println(br.readLine());
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
    public static void main(String[] args) {
        (new Cliente()).ejecutar();
    }
}
```

Detalles que restan puntos

Olvidar **flush()** (el cliente se queda esperando) · no comprobar **partes.length** (ArrayIndexOutOfBounds) · **parseInt** sin controlar NumberFormatException · mensajes distintos a los del enunciado · no cerrar el socket · guardar el estado compartido en una variable de instancia del hilo (cada cliente tendría el suyo).

Estado compartido entre clientes

Lo que deben ver **todos** los clientes (empleados, matrículas, libros…) se guarda en el **Servidor** (o en un atributo **static**) con una **Hashtable** (segura entre hilos) y se pasa a cada hilo: `new HiloServidor(socket, this)`. Las operaciones compuestas («si no existe, insértalo») van en métodos **synchronized**. Para cerrar hilos desde el servidor se guarda un **ArrayList** de hilos con id y un `matarHilo(id)` que llama a `cerrarSocket()`.

Ejemplo resuelto: el parcial real (fichaje de empleados)

Comandos: **ALTAEMPLEADO id nombre**, **ENTRADA id**, **SALIDA id**, **HORAS id**, **ESTADO id**. Reglas: no se puede ENTRADA con entrada abierta ni SALIDA sin entrada. Se usa la clase auxiliar **Empleado** (id, nombre, *Timestamp* de entrada o null, horas acumuladas) y **GestionFechas** (getHoraActual, getHorasEntre, getTexto).

```
public class HiloGestionFichaje extends Thread {
    // Listado de empleados (compartido por todos los hilos -> accesos synchronized)
    private static Hashtable<String, Empleado> empleados = new Hashtable<String, Empleado>();
    private Socket socket;
    private BufferedReader br;
    private PrintWriter pw;
    public HiloGestionFichaje(Socket socket) {
        this.socket = socket;
    }
    @Override
    public void run() {
        try {
            // Buffers de comunicacion con el cliente
            this.br = new BufferedReader(new InputStreamReader(this.socket.getInputStream()));
            this.pw = new PrintWriter(new OutputStreamWriter(this.socket.getOutputStream()));
            String linea;
            while ((linea = this.br.readLine()) != null) {
                String respuesta = this.procesar(linea);
                this.pw.println(respuesta);
                this.pw.flush();
            }
        } catch (IOException e) {
            System.out.println("Conexion finalizada");
        } finally {
            try { this.socket.close(); } catch (IOException e) { }
        }
    }
    private String procesar(String linea) {
        String[] partes = linea.trim().split(" ", 3);   // 3: el nombre completo puede llevar espacios
        String comando = partes[0].toUpperCase();
        switch (comando) {
        case "ALTAEMPLEADO":
            if (partes.length < 3) return "Error. Uso: ALTAEMPLEADO idEmpleado nombreCompleto";
            return this.altaEmpleado(partes[1], partes[2]);
        case "ENTRADA":
            if (partes.length < 2) return "Error. Uso: ENTRADA idEmpleado";
            return this.entrada(partes[1]);
        case "SALIDA":
            if (partes.length < 2) return "Error. Uso: SALIDA idEmpleado";
            return this.salida(partes[1]);
        case "HORAS":
            if (partes.length < 2) return "Error. Uso: HORAS idEmpleado";
```

```
            return this.horas(partes[1]);
        case "ESTADO":
            if (partes.length < 2) return "Error. Uso: ESTADO idEmpleado";
            return this.estado(partes[1]);
        default:
            return "Error. Comando no reconocido";
        }
    }
    private synchronized String altaEmpleado(String id, String nombre) {
        if (empleados.containsKey(id)) {
            return "Error. El empleado ya existe";
        }
        empleados.put(id, new Empleado(id, nombre));
        return "Ok. Empleado insertado";
    }
    private synchronized String entrada(String id) {
        Empleado e = empleados.get(id);
        if (e == null) return "Error. El empleado no existe";
        if (e.tieneEntradaAbierta()) {
            return "Error. El empleado ya ha registrado la entrada";
        }
        Timestamp ahora = GestionFechas.getHoraActual();
        e.setEntrada(ahora);
        return "Ok. Entrada registrada. " + GestionFechas.getTexto(ahora);
    }
    private synchronized String salida(String id) {
        Empleado e = empleados.get(id);
        if (e == null) return "Error. El empleado no existe";
        if (!e.tieneEntradaAbierta()) {
            return "Error. El empleado no tiene registrada entrada";
        }
        Timestamp ahora = GestionFechas.getHoraActual();
        e.sumaHoras(GestionFechas.getHorasEntre(e.getEntrada(), ahora));
        e.setEntrada(null);                              // cierra la entrada abierta
        return "Ok. Salida registrada. " + GestionFechas.getTexto(ahora);
    }
    private synchronized String horas(String id) {
        Empleado e = empleados.get(id);
        if (e == null) return "Error. El empleado no existe";
        return "Ok. " + e.getNombre() + " tiene trabajadas " + e.getHorasAcumuladas() + " horas.";
    }
    private synchronized String estado(String id) {
        Empleado e = empleados.get(id);
        if (e == null) return "Error. El empleado no existe";
        if (e.tieneEntradaAbierta()) {
            return "El empleado ha registrado entrada: " + GestionFechas.getTexto(e.getEntrada());
        }
        return "El empleado no ha registrado entrada";
    }
}
```

7 Errores típicos que caen en el examen

Las 4 preguntas de programación del parcial y cómo contestarlas

Si ves…	Causa	Cómo se arregla
BindException: Address already in use al ejecutar 2 veces	El puerto sigue ocupado por la ejecución anterior (servidor en while(true) que nunca cierra el ServerSocket).	Parar la anterior / cambiar puerto / cerrar el ServerSocket (try-with-resources).
Se lanzan 10 hilos con `new Thread().start()` y **no sale el texto** del Runnable	El Thread se crea **sin pasarle el Runnable**: su run() está vacío.	`new Thread(miHilo).start()`
5 hilos × 1000 incrementos, el contador **no llega a 5000**	**Condición de carrera** sobre una variable compartida; `contador++` no es atómico.	**synchronized** o AtomicInteger.

Si ves…	Causa	Cómo se arregla
IllegalThreadStateException en `Thread.start()`	Se llama a **start() varias veces sobre el mismo objeto** (un hilo solo se arranca una vez).	Crear un objeto nuevo en cada vuelta del bucle.
Un segundo cliente conecta pero **no recibe respuesta**	Servidor de un solo hilo, atrapado en el bucle del primer cliente.	**Un hilo por cliente**; el main solo hace accept().
ConnectException: Connection refused	Servidor no arrancado o puerto/IP distintos.	Arrancar primero el servidor; mismo puerto.
`t.run()` imprime el nombre **main**	run() es un método normal: se ejecuta en el hilo actual.	Usar **start()**.
`readLine()` devuelve **null** y el bucle imprime «null» sin parar	El cliente cerró la conexión.	`while ((l = br.readLine()) != null)`

Orden de salida con join()

Hilo que duerme 500 ms e imprime «A»; main imprime «B», hace `h.join()` y luego «C» → salida **B, A, C**. Sin join → **B, C, A**.

8 Autoevaluación (con respuestas)

Contesta tapando la columna derecha

Pregunta	Respuesta
1. ¿Qué es un hilo y para qué se usa?	Secuencia única de control de flujo de un programa. Tareas lentas, no paralizar el programa, evitar bloqueos, varias tareas a la vez.
2. Ventajas de la computación distribuida	Económica, compartición de recursos, escalabilidad y tolerancia a fallos.
3. ¿Qué significa «independientes» en un sistema distribuido?	No comparten memoria ni espacio de ejecución.
4. Distribuida vs paralela	Distribuida: muchos computadores en red. Paralela: varios procesadores a la vez para un programa.
5. ¿Qué es unicast y multicast?	Un proceso a otro / un proceso a varios.
6. Enviar/recibir bloqueante, ¿síncrono o asíncrono?	Bloqueante = síncrono. No bloqueante = asíncrono.
7. ¿Qué es un bloqueo indefinido y 2 soluciones?	Una petición que nunca se desbloquea. Temporizadores e hilos/proceso hijo.
8. Dos formas de crear un hilo y cómo se lanza	Extender Thread o implementar Runnable; se lanza con start() (Runnable: new Thread(obj).start()).
9. ¿Por qué falla start() dos veces sobre el mismo hilo?	Un hilo solo se arranca una vez → IllegalThreadStateException.
10. ¿Para qué sirve synchronized?	Solo un hilo a la vez en el método/bloque; evita condiciones de carrera.
11. ¿Qué es un socket y cómo se activa?	Punto de conexión entre dos procesos (IP + puerto). Con accept().
12. Pasos del servidor y del cliente	Servidor: socket, bind, listen, accept, leer/escribir. Cliente: socket, (bind), conectar, escribir/leer.

Pregunta	Respuesta
13. ¿Por qué se usa flush()?	Para forzar el envío del buffer; si no, el otro extremo no recibe y se bloquea.
14. ¿Qué es HTTP y 3 características?	Protocolo de transferencia de la web. URL, sin estado (cookies), 100 % de usuarios, genérico, no es HTML.
15. Estructura de un mensaje HTTP	Primera línea, cabeceras (campo:valor), línea en blanco CRLF, cuerpo.
16. Primera línea: petición vs respuesta	Petición: método + recurso + versión. Respuesta: versión + código de estado + frase.
17. Métodos HTTP importantes	GET, POST, PUT, DELETE.
18. Conexiones y puertos de FTP	Control (21) y datos (20).
19. PORT 127,0,0,1,16,0	IP 127.0.0.1, puerto 16·256 + 0 = 4096.
20. Códigos FTP 2yz / 3yz / 5yz	Correcto / intermedio positivo (falta info) / negativo definitivo.

9 Checklist del día antes

Marca todo antes de dormir

- ☐ Sé escribir de memoria un **Servidor** con accept() en bucle y un hilo por cliente.
- ☐ Sé escribir el **HiloServidor**: buffers, bucle de readLine, split, switch, println + flush, cerrar socket.
- ☐ Sé hacer un **cliente** con Scanner y salida por EXIT.
- ☐ Sé qué hace cada **error de hilos** del capítulo 7 y cómo se corrige.
- ☐ Sé explicar **synchronized** y por qué falla un contador compartido.
- ☐ Sé la **estructura de un mensaje HTTP**, sus características y los 4 métodos.
- ☐ Sé decir cuántas **conexiones** usa FTP y calcular un **PORT**.
- ☐ He hecho al menos un **ejercicio completo a mano** (fichaje, biblioteca, multas…) sin mirar la solución.

> **Consejo final**
> En el examen de programación puntúan **estructura y detalles**: buffers, flush, comprobaciones y mensajes exactos. Escribe primero el esqueleto del hilo (pasos 1-3 de la receta) y después rellena los comandos uno a uno: así siempre sumas puntos aunque no te dé tiempo a todo.